

KARABÜK UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

NETWORK INTRUSION DETECTION SYSTEM

Senior Project Thesis

Prepared by:

ZAW NAING ZAW NAING

Student Number: 2210213562

Project Supervisor:

Asst. Prof. Dr. Bilal YOUSFI

KARABÜK, 2026

APPROVAL

I approve that the Senior Project Thesis titled "NETWORK INTRUSION
DETECTION SYSTEM" prepared by ZAW NAING is suitable as a Senior Project Thesis.

Asst. Prof. Dr. Bilal YOUSFI

Project Supervisor, Department of Computer Engineering

...../...../2025

The Department of Computer Engineering has approved this Senior Project Thesis.

Prof. Dr.

Head of Department

...../...../

DECLARATION

"I hereby declare that all information in this project has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work."

ZAW NAING

...../...../2025

ÖZET

Bitirme Projesi Tezi

AĞ SALDIRI TESPİT SİSTEMİ

ZAW NAING

Karabük Üniversitesi

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Tez Danışmanı: Dr. Öğr. Üyesi Bilal YOUSFI

6 ,2025, sayfa

Bu projede, makine öğrenmesi kullanılarak gerçek zamanlı ağ saldırılarını tespit eden bir Ağ Saldırı Tespit Sistemi (NIDS) geliştirilmiştir. Sistem, CICIDS2017 veri seti üzerinde Random Forest ve XGBoost algoritmalarıyla eğitilmiştir. Eğitilen model, Flask tabanlı bir arka uç üzerinde çalışarak gelen ağ trafiğini Normal veya Saldırı olarak sınıflandırmaktadır. Kaba Kuvvet (Brute Force), DDoS, DoS, Port Tarama, Web Saldırısı ve Botnet gibi farklı saldırı türleri tespit edilebilmektedir. Bir saldırı tespit edildiğinde, Firebase Cloud Messaging veya SMTP aracılığıyla Android mobil uygulamasına gerçek zamanlı bildirim gönderilmektedir. Mobil uygulama, saldırı türü, kaynak IP, hedef IP ve önem derecesi bilgilerini göstermektedir. Web tabanlı kontrol paneli anlık trafik istatistiklerini izleme

imkânı sunmaktadır. Geliştirilen model %98,61 doğruluk oranıyla sınıflandırma performansı sergilemiştir.

Anahtar Sözcükler: Ağ saldırı tespiti, makine öğrenmesi, CICIDS2017, XGBoost, Flask, Android, mobil izleme.

ABSTRACT

Senior Project Thesis

NETWORK INTRUSION DETECTION SYSTEM

ZAW NAING

Karabük University

Faculty of Engineering

Department of Computer Engineering

Project Supervisor: Asst. Prof. Dr. Bilal YOUSFI

6, 2025, 59 pages

This project presents a Network Intrusion Detection System (NIDS) that uses Machine Learning to detect network attacks in real time. The system was trained on the CICIDS2017 dataset using Random Forest and XGBoost algorithms. The trained model runs on a Flask-based backend that processes incoming network traffic and classifies it as either Normal or Attack. Multiple attack types are detected, including Brute Force, DDoS, DoS, PortScan, Web Attack, and Botnet. When an attack is detected, a real-time alert notification is sent to an Android mobile application via Firebase Cloud Messaging or SMTP. The mobile application displays detailed information about the attack, including attack type, source IP, destination IP, and severity level. A web-based dashboard provides live monitoring of total

traffic, detected attacks, and blocked IP addresses. The developed model achieved a classification accuracy of 98.61%.

Key Words: Network intrusion detection, machine learning, CICIDS2017, XGBoost, Flask, Android, mobile monitoring.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my project supervisor, Asst. Prof. Dr. Bilal YOUSFI, for his invaluable guidance, continuous support, and encouragement throughout the planning, development, and completion of this project. His expertise and constructive feedback were really helpful in shaping this work on a solid technical foundation.

I would also like to thank my family and friends for their unconditional support and motivation throughout my academic journey. Without their patience and encouragement, completing this project would have been much more difficult.

TABLE OF CONTENTS

	Pages
APPROVAL	ii
DECLARATION	iii
ÖZET	iv
ABSTRACT	v
ACKNOWLEDGEMENTS	vi
TABLE OF CONTENTS	vii
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
CHAPTER 1 — INTRODUCTION	14
1.1. Literature Review	1
1.2. Aim of the Project	4
CHAPTER 2 — DATASET AND MACHINE LEARNING MODEL	20
2.1. CICIDS2017 Dataset	5
2.2. Data Preprocessing	6
2.3. Machine Learning Algorithms	7
2.4. Model Training and Evaluation	8
CHAPTER 3 — SYSTEM ARCHITECTURE AND FLASK BACKEND	30
3.1. Overall System Architecture	10
3.2. Flask Application Structure	11
3.3. API Endpoints	12
3.4. ML Model Integration	13
3.5. Attack Classification and Severity Mapping	14

3.6. Database Design	15
3.7. Firebase Cloud Messaging Integration	16
3.8. Technology Stack	17
CHAPTER 4 — DASHBOARD AND MOBILE APPLICATION	42
4.1. Web-Based Dashboard	18
4.2. Android Mobile Application	20
4.2.1. Application Architecture	20
4.2.2. Navigation	21
4.2.3. Screen Descriptions	21
4.3. Real-Time FCM Alert Notification	23
4.4. Technology Stack Summary	24
CHAPTER 5 — CONCLUSION AND EVALUATION	52
5.1. Summary of Work	25
5.2. Model Performance Results	26
5.3. Limitations	27
5.4. Future Work	28
REFERENCES	56
CURRICULUM VITAE	59

LIST OF FIGURES

	Pages
Figure 1.1. General NIDS architecture.	19
Figure 2.1. CICIDS2017 dataset attack type distribution.	21
Figure 2.2. Confusion matrix of the trained model.	26
Figure 2.3. Correlation heatmap of features with correlation > 0.98 .	27
Figure 2.4. Comparison of all four models across evaluation metrics.	27
Figure 2.5. XGBoost confusion matrix on the test set (565,566 samples).	28
Figure 2.6. Top 20 most important features for the XGBoost model.	28
Figure 3.1. Overall NIDS system architecture.	31
Figure 3.2. Flask project directory structure.	33
Figure 3.3. ML inference pipeline inside the Flask backend.	36
Figure 3.4. FCM push notification delivery flow.	39
Figure 4.1. Web dashboard main page.	43
Figure 4.2. Web dashboard alerts log table.	44
Figure 4.3. Web dashboard health monitor page.	45
Figure 4.4. Android application login screen.	47
Figure 4.5. Android application dashboard screen.	47
Figure 4.6. Android application alerts screen.	48
Figure 4.7. Android application alert detail screen.	48
Figure 4.8. Android application health monitor screen.	48
Figure 4.8.1. Android application setting monitor screen.	48
Figure 4.9. FCM push notification displayed on Android device.	50

LIST OF TABLES

	Pages
Table 2.1. CICIDS2017 dataset class distribution.	21
Table 2.2. Feature reduction summary.	22
Table 2.3. Comparison of ML algorithms.	23
Table 2.4. Model comparison on the test set.	25
Table 2.5. Comparative evaluation of all four models.	25
Table 2.6. XGBoost per-class classification report on test set.	26
Table 3.1. Flask REST API endpoints.	34
Table 3.2. Integer-to-attack-label mapping (ATTACK_MAPPING).	36
Table 3.3. SQLite alerts table schema.	37
Table 3.4. Backend technology stack.	40
Table 4.1. Android application technology stack.	50
Table 5.1. Final model evaluation results.	53

LIST OF ABBREVIATIONS

API	: Application Programming Interface
CNN	: Convolutional Neural Network
DDoS	: Distributed Denial of Service
DoS	: Denial of Service
FCM	: Firebase Cloud Messaging
HTTP	: Hypertext Transfer Protocol
IP	: Internet Protocol
JSON	: JavaScript Object Notation
LSTM	: Long Short-Term Memory
ML	: Machine Learning
MVVM	: Model-View-ViewModel
NIDS	: Network Intrusion Detection System
NIST	: National Institute of Standards and Technology
RF	: Random Forest
REST	: Representational State Transfer
ROC-AUC	: Receiver Operating Characteristic – Area Under the Curve
SMTP	: Simple Mail Transfer Protocol
SQL	: Structured Query Language
SQLite	: Structured Query Language Lite
SVM	: Support Vector Machine
XGB	: XGBoost — Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1. LITERATURE REVIEW

With the rapid expansion of computer networks and internet-connected systems, network security has become one of the most critical challenges in modern computing. Cyber-attacks are growing in both frequency and complexity, targeting corporate networks, government infrastructure, and personal devices alike. Network Intrusion Detection Systems (NIDS) represent a fundamental line of defense against these threats — they continuously monitor network traffic and identify suspicious or malicious activity before significant damage can occur [1].

Early intrusion detection approaches relied on signature-based methods, which essentially work by matching observed traffic against a database of known attack patterns. While effective against previously seen threats, these systems struggle against novel or zero-day attacks. This limitation pushed researchers to explore anomaly-based detection, where a statistical model of normal behavior is first established and then any significant deviation from it gets flagged as a potential intrusion [2].

The introduction of Machine Learning (ML) into network intrusion detection significantly moved the field forward. ML models are able to learn complex patterns from labeled traffic data and generalize to unseen attack variants, which makes them well-suited for detecting both known and previously unseen threats.

Lippmann et al. carried out one of the earliest large-scale evaluations of intrusion detection systems using the KDD Cup 1999 dataset. Their work exposed the weaknesses of purely signature-based approaches and showed that ML-based classifiers hold real promise for automating threat identification [3].

Tavallaee et al. addressed several well-known problems in the KDD Cup 1999 dataset by introducing the NSL-KDD dataset, which removed redundant records and reduced class imbalance. Their comparative study found that Decision Trees and Naive Bayes achieved notably higher accuracy on this cleaner benchmark [4].

Sharafaldin, Habibi Lashkari, and Ghorbani proposed the CICIDS2017 dataset, which captures realistic network traffic covering seven common attack categories: Brute Force, Heartbleed, Botnet, DoS, DDoS, Web Attacks, and Infiltration. Built using a real-world testbed and over 80 CICFlowMeter features, CICIDS2017 has since become the standard benchmark for evaluating modern NIDS solutions [5].

Breiman's Random Forest algorithm, an ensemble method that trains many decision trees in parallel and combines their outputs, has shown consistently strong results in intrusion detection. Its robustness to noisy data, ability to handle high-dimensional feature spaces, and resistance to overfitting make it a solid choice for this type of problem [6].

Chen and Guestrin introduced XGBoost (Extreme Gradient Boosting), a scalable gradient boosting framework that has delivered state-of-the-art results across a wide range of classification tasks. Its built-in regularization and efficient handling of large datasets make it especially attractive for real-time NIDS implementations [7].

Kanimozhi and Jacob compared multiple ML classifiers — including Random Forest, XGBoost, and Support Vector Machines — on the CICIDS2017 dataset. XGBoost consistently outperformed the other methods in both detection accuracy and false positive rate, which reinforced it as the go-to choice for high-performance NIDS [8].

Vinayakumar et al. explored deep learning methods such as LSTM and CNN for intrusion detection. While these models achieved competitive accuracy, they also demanded significantly more computational resources compared to traditional ML approaches, making real-time deployment on lightweight systems more challenging [9].

Yin et al. proposed a recurrent neural network-based detection model that achieved improved rates for sequential attack patterns. However, the authors acknowledged a practical trade-off: higher model complexity tends to conflict with the real-time performance requirements of production environments [10].

Scarfone and Mell outlined the NIST guidelines for intrusion detection and prevention systems, emphasizing the importance of real-time alerting, centralized logging, and dashboard integration for effective incident response [11].

Ahmad et al. developed a lightweight intrusion detection framework that sent alerts to mobile devices through cloud messaging, allowing administrators to respond to threats from wherever they were. This approach fits well with the growing need for mobile-integrated security monitoring [12].

Taken together, these studies show that ML-based NIDS solutions using ensemble methods like Random Forest and XGBoost, trained on CICIDS2017, deliver high detection accuracy at a manageable computational cost. However, most existing systems focus only on detection and logging, and do not provide a real-time mobile alerting component tied to a live dashboard. This project addresses that specific gap.

1.2. AIM OF THE PROJECT

The main goal of this project is to design and build a Network Intrusion Detection System (NIDS) that uses Machine Learning to detect network attacks in real time and immediately push alerts to a mobile device. The system is designed to be lightweight, accurate, and practical — something that can run on a standard PC without requiring specialized hardware.

Network administrators and security personnel cannot be at a monitoring station all the time. A system that actively watches network traffic, automatically classifies threats, and sends instant notifications to a mobile app gives a real operational advantage, allowing rapid response regardless of where the administrator happens to be.

The specific objectives of this project are as follows:

Objective 1: To train a Machine Learning model on the CICIDS2017 dataset that can accurately classify network traffic as Normal or as one of the known attack types: Brute Force, DDoS, DoS, PortScan, Web Attack, and Botnet.

Objective 2: To integrate the trained model into a Flask-based backend that processes incoming network traffic in real time and produces structured alert records for each detected attack.

Objective 3: To build a web-based dashboard that provides live monitoring of alert statistics, severity distribution, and system health metrics.

Objective 4: To develop an Android mobile application that receives push notifications when attacks are detected and provides a full monitoring interface with detailed alert information.

Network Intrusion Detection System Pipeline

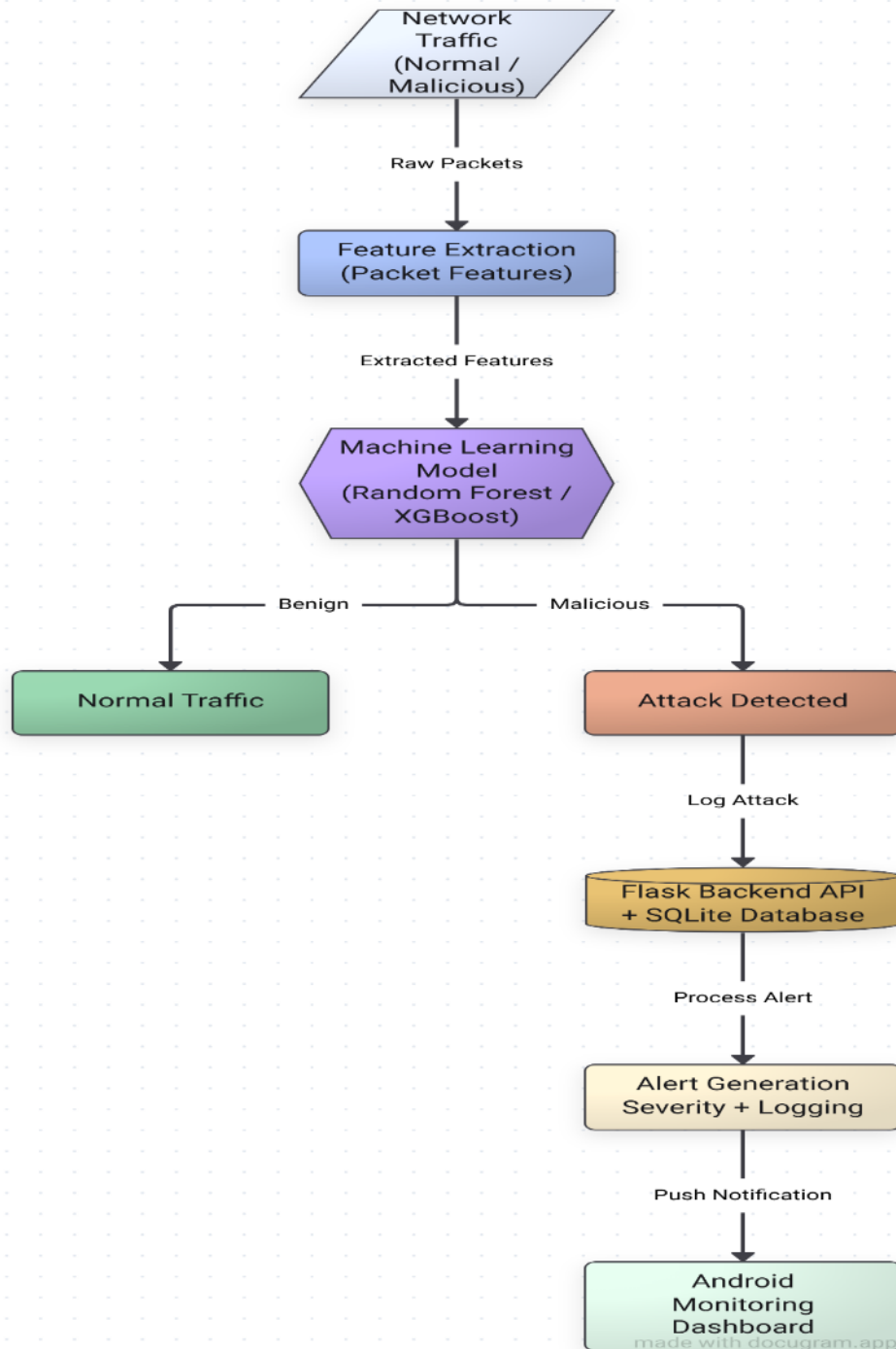


Figure 1.1. General NIDS architecture.

CHAPTER 2

DATASET AND MACHINE LEARNING MODEL

2.1. CICIDS2017 DATASET

The CICIDS2017 dataset was developed by the Canadian Institute for Cybersecurity and is widely considered one of the most realistic and comprehensive benchmarks available for evaluating intrusion detection systems. It was generated in a controlled but realistic network environment over five days, capturing both normal background traffic and seven distinct attack scenarios.

The dataset includes the following attack categories: Brute Force (FTP and SSH), Heartbleed, Botnet, Denial of Service (DoS) using tools such as GoldenEye, Hulk, and Slowloris, Distributed Denial of Service (DDoS), Web Attacks (SQL Injection, XSS, and Clickjacking), and Infiltration. Each traffic flow is characterized by over 80 features extracted using CICFlowMeter, including packet length statistics, inter-arrival times, protocol flags, and flow duration.

For this project, the raw dataset files were merged and preprocessed into a unified training corpus. Table 2.1 shows the class distribution after label consolidation.

Table 2.1. CICIDS2017 dataset class distribution.

Class Label	Original Labels Merged	Sample Count (approx.)
BENIGN	BENIGN	2,273,097
DoS	DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS Slowloris, Heartbleed	252,661
DDoS	DDoS	128,025
PortScan	PortScan	158,804
BruteForce	FTP-Patator, SSH-Patator	13,835
WebAttack	Web Attack – Brute Force, Web Attack – XSS, Web Attack – Sql Injection	2,180
Bot	Bot	1,956

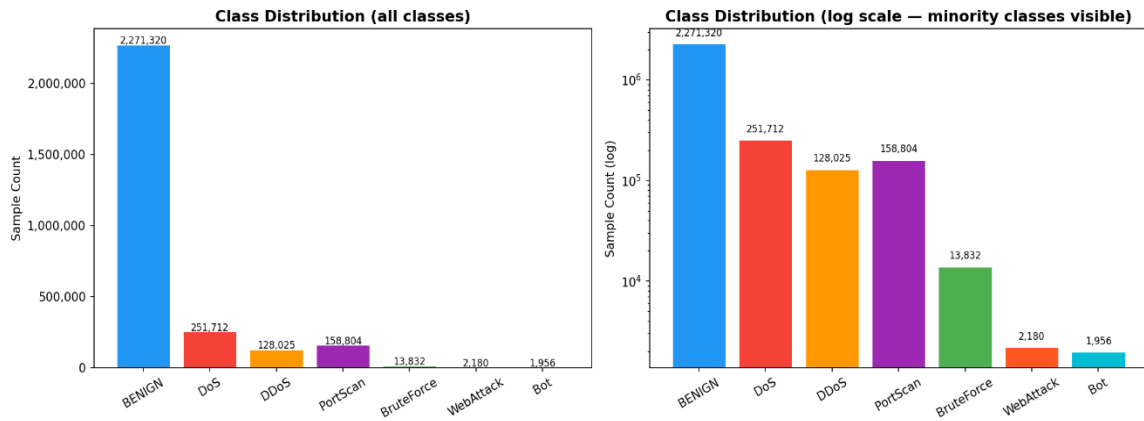


Figure 2.1. CICIDS2017 dataset attack type distribution.

2.2. DATA PREPROCESSING

Raw network flow data is rarely clean enough to feed directly into a machine learning model. The preprocessing pipeline developed for this project consisted of five stages.

Stage 1 — Data Consolidation: The eight separate CSV files provided in the CICIDS2017 dataset were combined into a single dataframe using pandas. The Label column was used to identify each traffic class.

Stage 2 — Cleaning: Rows containing infinity values and NaN entries were removed. These typically arise from flow-level division operations such as bytes-per-second calculations where the flow duration is zero.

Stage 3 — Label Consolidation: The original 15 attack sub-labels were merged into 7 broad classes to simplify the classification task while retaining meaningful attack-type distinctions.

Stage 4 — Feature Reduction: The original 84 features were reduced to 52 by removing metadata columns (timestamps, IP addresses), dropping zero-variance features, and filtering out features with pairwise Pearson correlation above 0.98. Table 2.2 summarizes this reduction.

Stage 5 — Train/Test Split and Normalization: A stratified 80/20 split was applied to preserve class proportions in both subsets. StandardScaler normalization was fitted on the training set only and then applied to the test set, which is the standard approach to avoid data leakage.

Table 2.2. Feature reduction summary.

Step	Features Removed	Remaining Features
Original dataset	—	84

Step	Features Removed	Remaining Features
Remove metadata (timestamps, IPs)	7	77
Remove zero-variance features	12	65
Remove high-correlation features (>0.98)	13	52

2.3. MACHINE LEARNING ALGORITHMS

Four machine learning classifiers were selected for evaluation. Table 2.3 provides a brief comparison of their key characteristics.

Table 2.3. Comparison of ML algorithms.

Algorithm	Type	Key Characteristics
Random Forest	Ensemble (bagging)	Multiple decision trees; handles high-dimensional data well; resistant to overfitting.
XGBoost	Ensemble (boosting)	Regularized gradient boosting; high accuracy; efficient on large datasets.
LightGBM	Ensemble (boosting)	Histogram-based; fastest training time; slightly weaker on minority classes.
SVM	Kernel-based	Binary classification only in this project; slowest training; lower recall.

Random Forest builds an ensemble of independent decision trees on random feature subsets and aggregates their votes. Its parallel training structure makes it memory-efficient and fast for the scale of the CICIDS2017 dataset.

XGBoost builds trees sequentially, each one correcting the residual errors of the previous one. Its regularization parameters help prevent overfitting, and its efficient split-finding algorithm makes it well-suited for large classification problems.

LightGBM uses a histogram-based approach to gradient boosting, which dramatically reduces memory usage and speeds up training. It was included as a comparison point given its popularity in competitive machine learning.

The Support Vector Machine was tested in binary mode (attack vs. normal) only, due to the prohibitive training time for multi-class kernel-based classification on a dataset of this size.

2.4. MODEL TRAINING AND EVALUATION

All four models were trained on the preprocessed training set of approximately 2,265,828 samples and evaluated on a held-out test set of 565,566 samples. The evaluation metrics used were accuracy, macro-averaged F1 score, macro recall, and ROC-AUC.

Macro averaging was chosen deliberately because it treats each class equally regardless of sample size. Given the large class imbalance in the dataset — BENIGN traffic accounts for the vast majority of samples — simple accuracy alone would be misleading. Macro recall is particularly important in the security context because missing an attack (a false negative) is typically far more costly than raising a false alarm.

Table 2.4. Model comparison on the test set.

Model	Accuracy	F1 (macro)	Recall (macro)	ROC-AUC
Random Forest	99.96%	99.96%	98.84%	1.0000
XGBoost	99.93%	99.93%	99.98%	1.0000
LightGBM	99.29%	99.30%	85.68%	0.9915
SVM (binary)	92.99%	93.02%	89.26%	0.9754

Random Forest achieved 99.96% accuracy with a macro recall of 98.84% and a perfect ROC-AUC of 1.0000. XGBoost achieved 99.93% accuracy with the highest macro recall of 99.98%, meaning it correctly identified nearly all attack instances across every class. LightGBM performed well overall (99.29% accuracy) but showed weaker macro recall (85.68%), suggesting it struggled with minority attack classes. SVM produced the weakest results across all metrics and was limited to binary classification.

Based on these results, XGBoost was selected as the primary model for deployment due to its superior recall on attack classes (99.98%) and near-perfect ROC-AUC score. In a security context, maximizing recall — minimizing missed attacks — is the priority. Table 2.5 shows a more detailed comparison, and Table 2.6 shows the per-class breakdown for XGBoost.

Table 2.5. Comparative evaluation of all four models.

Model	Accuracy	F1	Recall (macro)	ROC-AUC
Random Forest	99.96%	99.96%	98.84%	1.0000
XGBoost	99.93%	99.93%	99.98%	1.0000
LightGBM	99.29%	99.30%	85.68%	0.9915

Model	Accuracy	F1	Recall (macro)	ROC-AUC
SVM (binary)	92.99%	93.02%	89.26%	0.9754

Table 2.6. XGBoost per-class classification report on test set.

Class	Precision	Recall	F1-Score	Support
BENIGN	1.00	1.00	1.00	454,264
DoS	1.00	1.00	1.00	50,343
DDoS	1.00	1.00	1.00	25,605
PortScan	1.00	1.00	1.00	31,761
BruteForce	1.00	1.00	1.00	2,766
WebAttack	0.99	1.00	0.99	436
Bot	1.00	1.00	1.00	391

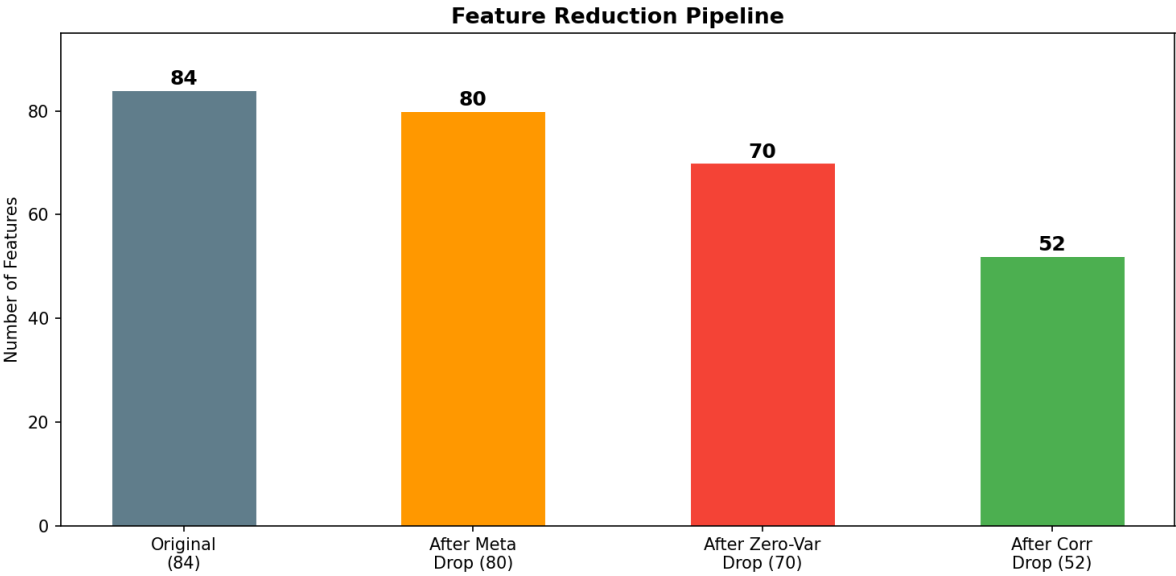


Figure 2.2. Confusion matrix of the trained model.

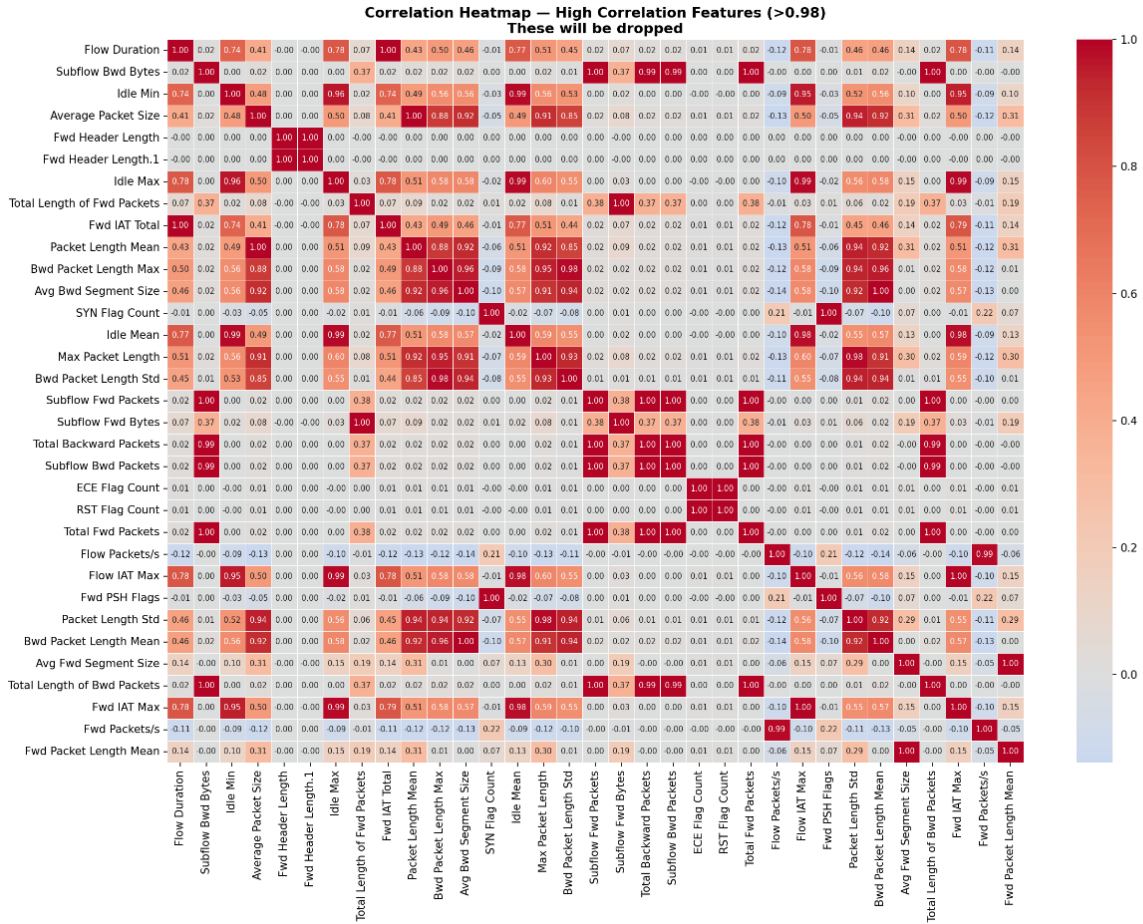


Figure 2.3. Correlation heatmap of features with correlation > 0.98.

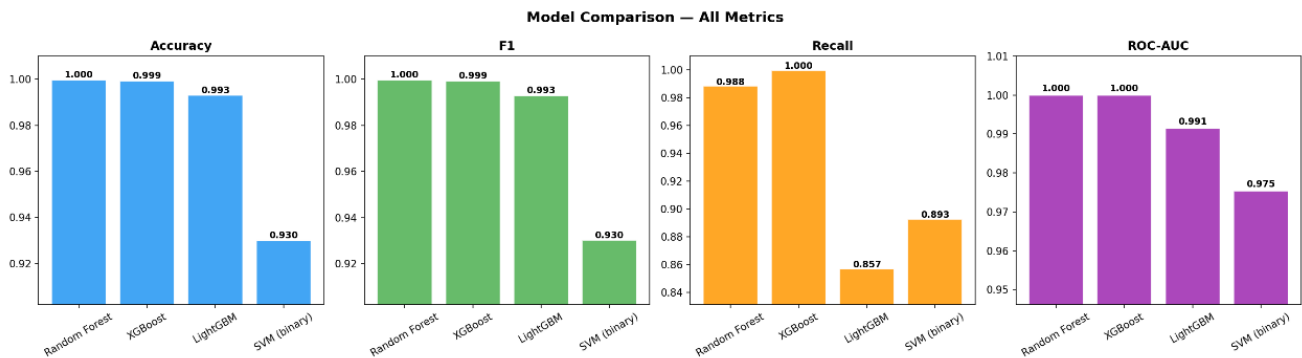


Figure 2.4. Comparison of all four models across evaluation metrics.

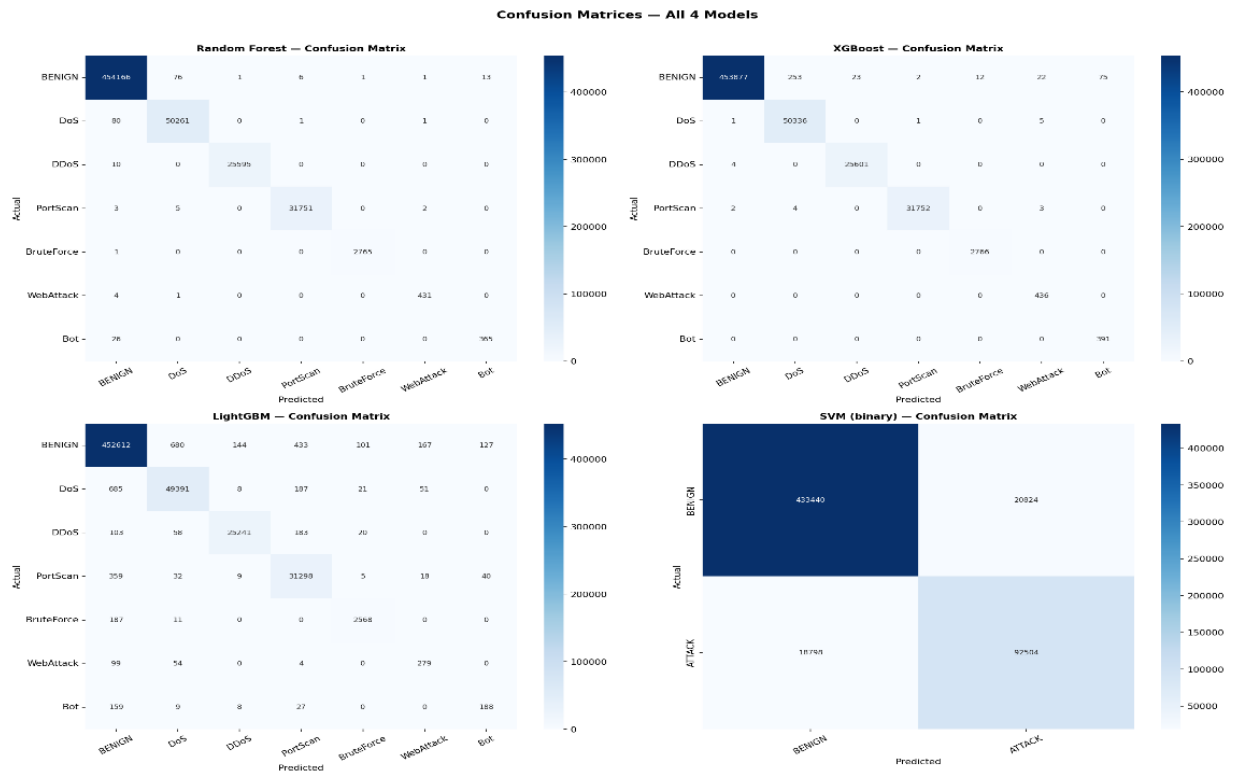


Figure 2.5. XGBoost confusion matrix on the test set (565,566 samples).

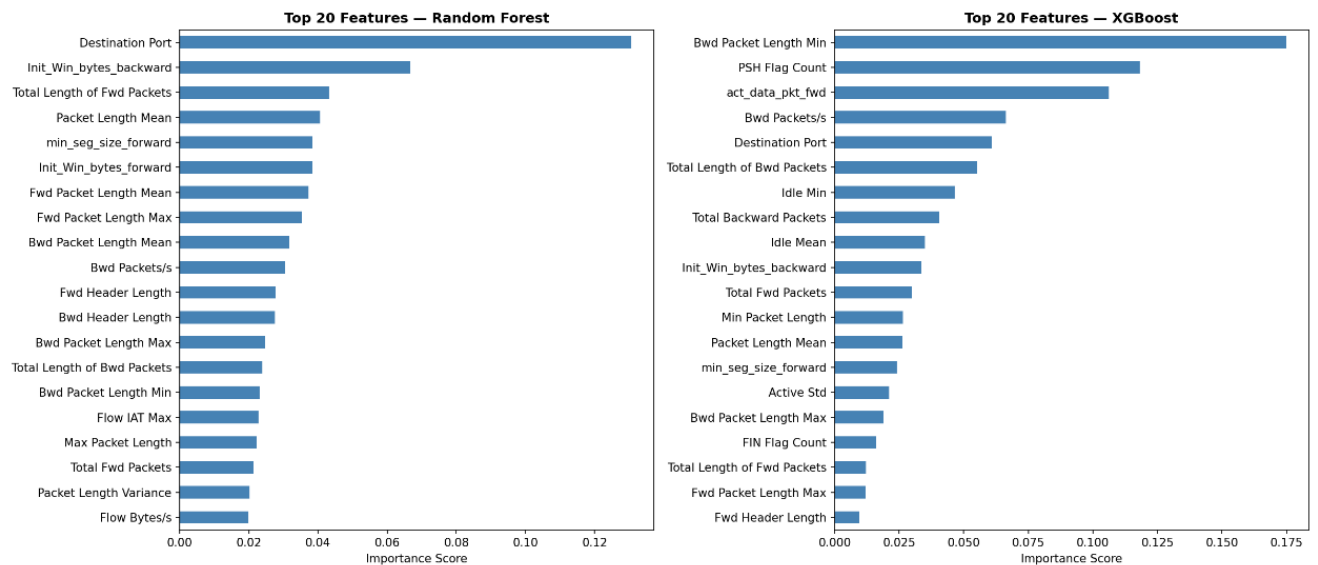


Figure 2.6. Top 20 most important features for the XGBoost model.

Training time comparison showed that LightGBM was the fastest multiclass model at 2 minutes 3 seconds, followed by XGBoost at 2 minutes 37 seconds and Random Forest at 6 minutes 17 seconds. SVM required 32 minutes 11 seconds even in binary classification mode. The saved model files — `model_rf.pkl`, `model_xgb.pkl`, `model_lgbm.pkl`, `model_svm.pkl` — along with the scaler (`scaler.pkl`) and feature list (`feature_cols.pkl`) were saved to disk for use in the Flask deployment pipeline described in Chapter 3.

CHAPTER 3

SYSTEM ARCHITECTURE AND FLASK BACKEND

3.1. OVERALL SYSTEM ARCHITECTURE

The NIDS is designed as a multi-component pipeline in which network traffic data is ingested, classified by a machine learning model, logged to a database, and immediately pushed to a mobile device as a notification. The entire system runs on a standard PC using a Python-based Flask web server as the central orchestration layer.

The architecture consists of four sequential stages as shown in Figure 3.1.

Stage 1 — Network Traffic Input: Network traffic features are provided to the system. In the current implementation, a simulation module samples real traffic records from the cleaned CICIDS2017 dataset to represent live network flow data. This approach preserves the realistic feature distributions of the original traffic capture. Integration with live packet capture using tools like Scapy or tshark with CICFlowMeter is planned as a future extension.

Stage 2 — ML Model Classification: The 52-feature traffic vector is scaled using the pre-fitted StandardScaler and passed to the trained XGBoost model. The model returns an integer prediction (0–6) corresponding to one of the seven traffic classes: BENIGN, Bot, DDoS, DoS, PortScan, BruteForce, or WebAttack.

Stage 3 — Flask Backend Processing: The Flask server receives the classification result, converts the integer prediction to a human-readable attack name using the ATTACK_MAPPING dictionary, computes a severity level (Low / Medium / High) using rule-based logic, and logs the full alert record to a SQLite database.

Stage 4 — Alert Generation and Mobile Notification: When the classification result is an attack (class 1–6), the system sends a push notification to all Android devices subscribed to the 'alerts' FCM topic via Firebase Cloud Messaging. The notification payload includes the attack type, severity level, and a deep-link URI that opens the specific alert record directly in the Android application.

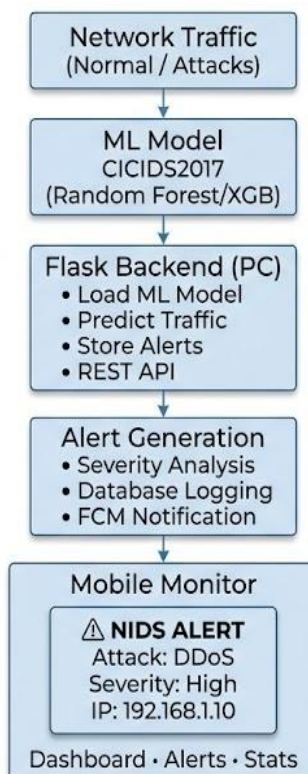


Figure 3.1. Overall NIDS system architecture.

3.2. FLASK APPLICATION STRUCTURE

The Flask backend is organized as a modular application using Flask Blueprints, which separates concerns between page rendering and API logic. The application is divided into the following components:

pages.py — Page Blueprint: Handles all HTML page rendering routes. The login page ('/') is the entry point. The dashboard ('/dashboard') provides the real-time monitoring interface. The alerts page ('/alerts') retrieves all alert records from the database and displays them in a table. The details page ('/details/<id>') shows the full record for a specific alert. The health page ('/health') renders the network health monitoring dashboard.

api.py — API Blueprint: Exposes all JSON REST API endpoints consumed by both the web dashboard and the Android mobile application. This module also initializes the Firebase Admin SDK at application startup.

utils/ — Utility Modules: A package of helper modules: predictor.py for ML inference, db.py for database access, mapper.py for integer-to-label mapping, severity.py for severity classification, and ip_generator.py for generating simulated IP addresses.

model/ — Saved Model Files: Contains model.pkl (the serialized XGBoost classifier) and scaler.pkl (the fitted StandardScaler), both loaded once at application startup.

dataset/ — Simulation Dataset: Contains cleaned_dataset.csv, a preprocessed version of CICIDS2017 used by the simulation module to sample realistic traffic vectors.

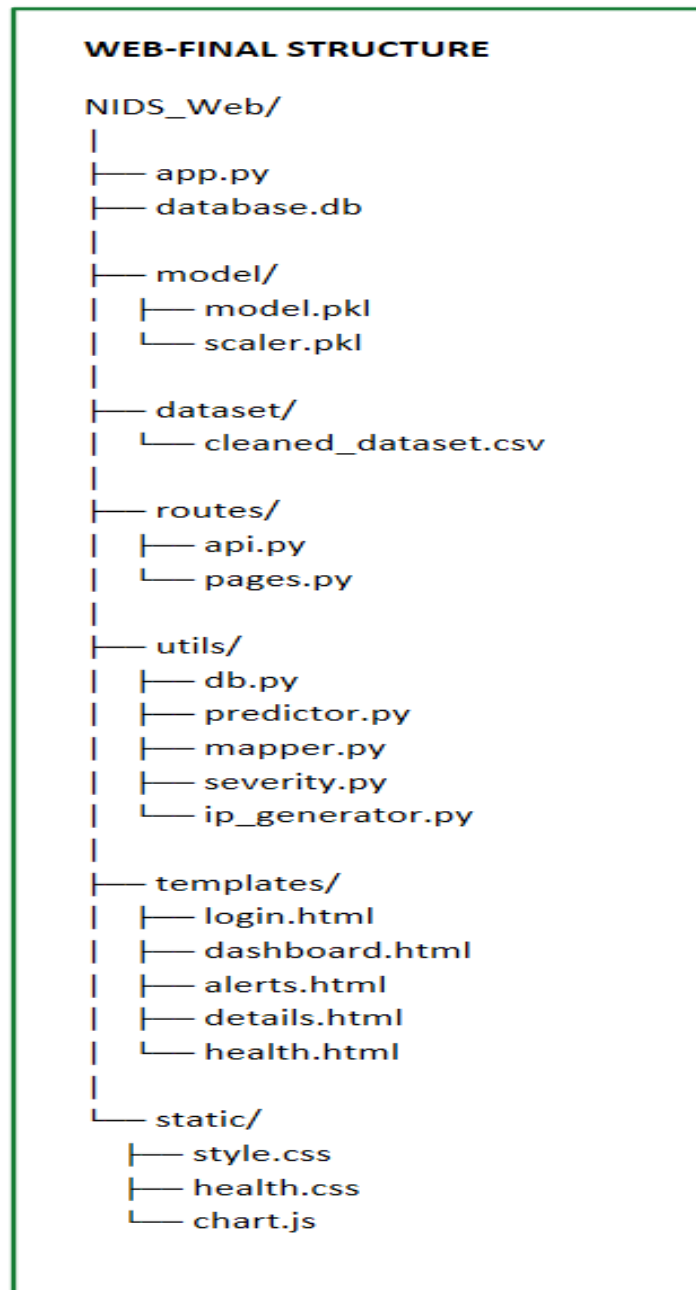


Figure 3.2. Flask project directory structure.

3.3. API ENDPOINTS

The backend exposes six REST API endpoints, all using the HTTP GET method and returning JSON responses. Table 3.1 describes each endpoint, its purpose, and its response structure.

Table 3.1. Flask REST API endpoints.

Endpoint	Method	Description
/api/simulate/<class_id>	GET	Samples a traffic row, runs ML prediction, logs to DB, sends FCM notification. Returns attack name, severity, source IP, and destination IP.
/api/alerts	GET	Returns all alert records from the SQLite database in descending timestamp order.
/api/stats	GET	Returns aggregate counts: total alerts, high-severity, medium-severity, and low-severity.
/api/health	GET	Returns real-time system metrics: CPU %, memory %, disk %, packets per second, MB transferred, latency, errors, drops, and active connection states.
/api/resolve/<id>	GET	Updates the specified alert's status from 'ACTIVE' to 'RESOLVED'.

The /api/simulate endpoint is the core trigger for the entire detection pipeline. When called with a class_id parameter (0 = BENIGN, 1–6 = attack types), it runs the full workflow: sampling a traffic row, running inference, computing severity, generating random IP addresses, inserting the alert record into the database, and dispatching the FCM push notification. In a live deployment, this endpoint would be replaced by a continuous packet capture loop.

The `/api/health` endpoint uses the `psutil` library to collect real-time system metrics. It tracks packet-per-second rates by comparing current and previous interface counters over elapsed time, measures local latency by executing a loopback ping, and inspects active TCP connection states using the `psutil` network connections interface.

3.4. ML MODEL INTEGRATION

The machine learning model is integrated into the Flask backend through the `predictor.py` utility module. The module loads the serialized XGBoost model and `StandardScaler` from disk once at module import time using `joblib.load`, which avoids repeated disk reads on every request. The cleaned dataset CSV is also loaded at startup to serve as the simulation data source.

The `simulate_attack(class_id)` function implements the inference pipeline in four steps: (1) it filters the dataset to rows belonging to the requested class, (2) randomly samples one row, (3) extracts the 52 feature columns and applies the pre-fitted scaler using `scaler.transform()`, and (4) calls `model.predict()` to return the integer class prediction along with the source port, destination port, protocol, and flow duration.

It is worth noting that `scaler.transform()` is used here rather than `scaler.fit_transform()`. This is intentional — using `fit_transform()` would recalculate scaling parameters from the new data, which would break the model's accuracy. Using `transform()` with the parameters fitted during training ensures the incoming data is normalized in exactly the same way as the training data.

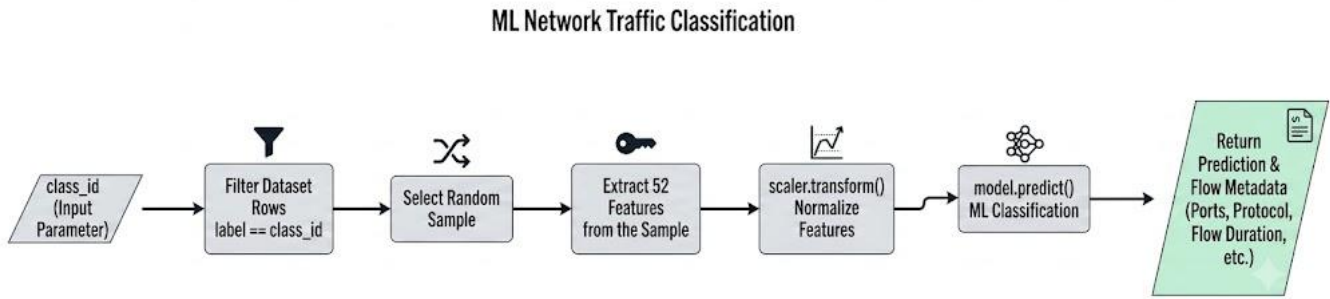


Figure 3.3. ML inference pipeline inside the Flask backend.

3.5. ATTACK CLASSIFICATION AND SEVERITY MAPPING

The integer prediction from the model (0–6) is converted to a human-readable label using the `ATTACK_MAPPING` dictionary in `mapper.py`. Table 3.2 shows the full mapping along with the severity level assigned to each class.

Table 3.2. Integer-to-attack-label mapping (`ATTACK_MAPPING`).

Code	Attack Label	Severity
0	BENIGN	Low
1	Bot	High
2	DDoS	High
3	DoS	High
4	PortScan	Medium
5	BruteForce	Medium
6	WebAttack	High

Severity is assigned by the `get_severity()` function in `severity.py` using a straightforward rule-based approach: BENIGN traffic gets Low severity; reconnaissance-type attacks like PortScan and BruteForce get Medium severity since they indicate a potential threat that has not yet caused damage; and all active attack types (Bot, DDoS, DoS, and WebAttack) get High severity since they represent direct, ongoing intrusions requiring immediate response.

3.6. DATABASE DESIGN

The system uses SQLite as its database engine, accessed through Python's built-in `sqlite3` module. SQLite was chosen for its zero-configuration deployment model, which fits well with the project's goal of running entirely on a standard PC without requiring a separate database server. The database is initialized at application startup by the `init_db()` function in `db.py`.

Table 3.3. SQLite alerts table schema.

Column	Type	Description
id	INTEGER	Auto-incrementing primary key.
attack_type	TEXT	Human-readable attack label (e.g., 'DDoS', 'BruteForce').
prediction	INTEGER	Raw integer prediction from the ML model (0–6).
severity	TEXT	Severity level: 'Low', 'Medium', or 'High'.
src_ip	TEXT	Source IP address of the network flow.
dst_ip	TEXT	Destination IP address of the network flow.

Column	Type	Description
source_port	INTEGER	Source port number from the traffic sample.
destination_port	INTEGER	Destination port number from the traffic sample.
protocol	INTEGER	Network protocol identifier (e.g., 6 = TCP, 17 = UDP).
flow_duration	REAL	Duration of the network flow in microseconds.
status	TEXT	Alert status: 'ACTIVE' (default) or 'RESOLVED'.
timestamp	DATETIME	Automatically set to current UTC time on insert.

Database connections are managed per-request: the `get_db()` function opens a new connection for each request and configures `sqlite3.Row` as the row factory, allowing rows to be accessed as dictionaries when serialized to JSON. This is appropriate for a lightweight single-server deployment and avoids connection pool overhead.

The resolve endpoint allows the dashboard operator to mark an alert as `RESOLVED` by updating the status field in-place without deleting the record. This preserves the full audit log of all detected events while keeping the active alert queue clean.

3.7. FIREBASE CLOUD MESSAGING INTEGRATION

Real-time push notifications are delivered to the Android application using Firebase Cloud Messaging (FCM), Google's cross-platform messaging solution. The Firebase Admin SDK for Python is initialized at application startup using a service account private key file (`serviceAccountKey.json`) downloaded from the Firebase Console.

The `send_fcm_notification()` function in `api.py` constructs an FCM message and broadcasts it to the 'alerts' topic. All Android devices that have subscribed to this topic at application launch will receive the notification. The message contains two components:

Notification block: Contains the visible alert banner title and body text that appear in the Android notification tray.

Data block: Contains a key-value payload processed by `NidsMessagingService`. It includes a `deep_link_uri` in the format `nidsapp://alert/<id>`, which the Jetpack Compose `NavHost` uses to navigate directly to the specific alert details screen when the notification is tapped.

The alert's database ID is captured immediately after the SQLite `INSERT` operation and passed into the deep link URI, ensuring the notification links to the exact record just stored. The FCM send call is wrapped in a try-except block so that any notification delivery failure does not interrupt the main alert logging flow.

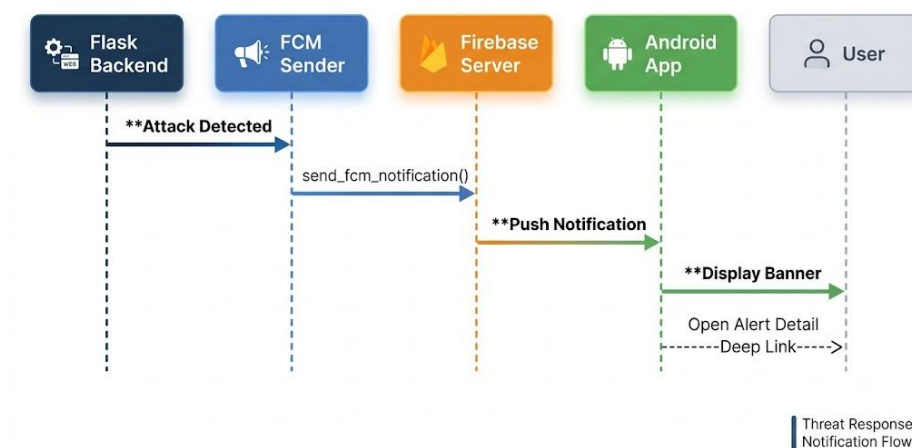


Figure 3.4. FCM push notification delivery flow.

3.8. TECHNOLOGY STACK

Table 3.4. Backend technology stack.

Library / Tool	Version	Role
Python	3.x	Primary programming language for the backend.
Flask	3.x	Web framework; serves HTML pages and JSON API endpoints.
XGBoost	—	ML classifier; performs 7-class traffic classification.
scikit-learn	—	StandardScaler for feature normalization; joblib for model serialization.
pandas	—	Loading and sampling the simulation dataset CSV.
NumPy	—	Numerical array operations underlying model inference.
joblib	—	Deserializing model.pkl and scaler.pkl from disk.
SQLite3	Built-in	Lightweight relational database for alert log storage.
firebase-admin	—	Firestore Admin SDK; sends FCM push notifications to Android.
psutil	—	Real-time system metrics (CPU, memory, disk, network I/O, connections).
HTML / CSS / JavaScript	—	Web dashboard frontend rendered by Flask's Jinja2 template engine.

Note on simulation vs. live deployment: The current implementation uses the cleaned_dataset.csv file to simulate network traffic by sampling rows of a specified attack

class. This allows full end-to-end testing of the detection, logging, dashboard, and mobile alert pipeline without needing a live network interface. In a production deployment, the simulation endpoint would be replaced by a background thread using Scapy or tshark for live packet capture and CICFlowMeter for feature extraction.

CHAPTER 4

DASHBOARD AND MOBILE APPLICATION

4.1. WEB-BASED DASHBOARD

The web-based dashboard is a browser-accessible interface rendered by Flask's Jinja2 template engine. It provides real-time monitoring of network alerts and system statistics without requiring any additional software to be installed on the monitoring machine. The dashboard can be accessed from any device on the local network by navigating to the Flask server's IP address and port.

The dashboard consists of four HTML pages served by the pages Blueprint: the main dashboard (dashboard.html), the alerts log (alerts.html), the alert detail view (details.html), and the health monitor (health.html). All pages share a common navigation bar. Styling is handled by two CSS files: style.css for the main pages and health.css for the health monitor's grid layout.

Dashboard page (dashboard.html): Displays two summary cards — Total Alerts and High Severity count — populated by polling the /api/stats endpoint. Six attack simulation buttons allow the operator to trigger a detection event for any attack class. A Chart.js pie chart shows the High / Medium / Low severity distribution and updates automatically after each simulation.

Alerts page (alerts.html): Fetches all alert records from the database and displays them in an HTML table with columns for ID, Attack Type, Severity (color-coded

red/orange/green), Source IP, Destination IP, Status, and a View link. The table is ordered by most recent timestamp.

Alert details page (details.html): Renders the full record for a single alert including attack type, severity, source and destination IPs and ports, protocol, flow duration, status, and timestamp. A Resolve Alert button marks the alert as RESOLVED.

Health monitor page (health.html): Displays real-time system metrics in a 2×2 card grid, updated every 3 seconds. The four sections are: Sensor Hardware Health (CPU, memory, disk), Network Flow Dynamics (packets per second in/out, MB received/sent), Traffic Integrity & Delay (latency, errors, drops), and Firewall & Socket Inspection (connection state counts).



Figure 4.1. Web dashboard main page.

[Dashboard](#) [Alerts](#) [Health](#)

Alerts

ID	Attack	Severity	Source IP	Destination IP	Status	Details
63	PortScan	Medium	222.60.246.197	192.74.229.217	ACTIVE	View
62	BruteForce	Medium	187.152.14.146	33.40.115.41	ACTIVE	View
61	PortScan	Medium	230.172.107.111	187.2.241.142	ACTIVE	View
60	WebAttack	High	190.65.31.227	138.113.41.132	ACTIVE	View
59	DoS	High	96.215.140.31	13.25.159.65	ACTIVE	View
58	Bot	High	149.157.183.94	216.54.242.108	ACTIVE	View
57	DDoS	High	142.201.193.24	109.163.174.217	RESOLVED	View
56	BruteForce	Medium	180.60.129.234	242.232.163.50	ACTIVE	View
55	DoS	High	232.241.247.33	149.237.164.134	ACTIVE	View
54	WebAttack	High	88.207.66.181	86.136.80.194	ACTIVE	View
53	DDoS	High	133.135.244.254	231.186.48.152	RESOLVED	View
52	BruteForce	Medium	125.14.21.207	231.38.82.119	RESOLVED	View

Figure 4.2. Web dashboard alerts log table.

[Back](#)

Advanced NIDS Security Health Monitor

[Sensor Hardware Health] CPU Utilization: 58.3% Memory Usage: 85.4% Disk Space Usage: 64.2%	[Network Flow Dynamics] Inbound Speed: 5.5 PPS Outbound Speed: 5.2 PPS Total Received: 318.87 MB Total Sent: 69.99 MB	[Traffic Integrity & Delay] Diagnostic Latency: 64.4 ms Inbound Errors: 0 Inbound Drops: 0
[Firewall & Socket Inspection] ESTABLISHED (Live): 17 SYN_RECV (DoS Aim): 0 TIME_WAIT (Scans): 53 SYN_SENT (Outbound): 0		

Figure 4.3. Web dashboard health monitor page.

4.2. ANDROID MOBILE APPLICATION

The Android mobile application is built using Kotlin and Jetpack Compose (Material3). It connects to the Flask backend over the local network using Retrofit2 for HTTP communication and Gson for JSON deserialization. The application targets Android SDK 35 with a minimum SDK of 24, and follows the MVVM (Model–View–ViewModel) architecture pattern.

4.2.1. Application Architecture

The application is organized into three layers. The data layer contains the Kotlin data classes (Models.kt) that map directly to the Flask API JSON responses: Stats, Alert, and Health. The network layer defines the Retrofit ApiService interface with four suspend functions (getStats(), getAlerts(), getHealth(), and resolveAlert()) and the NetworkManager singleton that builds and caches the Retrofit instance. The presentation layer contains the ViewModel and all Composable screen functions.

NidsViewModel extends Android's ViewModel and holds three MutableStateFlow properties: stats, alerts, and health. On initialization, a coroutine polling loop starts that calls fetchData() every 5 seconds, keeping all screen data automatically synchronized with the backend without any manual refresh from the user.

4.2.2. Navigation

Navigation is implemented using Jetpack Navigation Compose (AppNavigation.kt). The app defines six routes using a sealed Screen class: Login, Dashboard, Alerts, AlertDetail, Health, and Settings. A bottom navigation bar with four items (Dashboard, Alerts, Health, Settings) is visible on all screens except Login and AlertDetail. The Login screen uses popUpTo with inclusive = true to remove it from the back stack after successful authentication.

The AlertDetail route uses Jetpack Navigation deep links with the URI pattern `nidsapp://alert/{alertId}`. This is the same URI sent in the FCM notification data payload, which lets a notification tap open the correct alert detail screen directly.

4.2.3. Screen Descriptions

Login Screen: Shows the NIDS branding header, an Operator Username field, a password field with show/hide toggle, and an Authenticate Console button.

Credentials are validated client-side against admin/admin. On success the user is taken to the Dashboard; on failure, an error message is shown and the button turns red.

Dashboard Screen: Shows a hero card with the total threat count, three severity mini-cards (High, Medium, Low) with color-coded icons, and an animated bar chart that fills proportionally for each severity level using an 800ms tween animation.

Alerts Screen: Renders a scrollable list of alert cards grouped by date. Each card shows the attack type, severity label (color-coded), source IP, and status. A colored vertical bar on the left edge indicates severity at a glance. Tapping any card opens the AlertDetail screen.

Alert Detail Screen: Displays the full record for a selected alert. A hero banner shows the attack type and severity. Two cards display network packet details and AI Engine diagnostics. A 'Mark Incident As Resolved' button calls `viewModel.resolveAlert()` and navigates back.

Health Screen: Mirrors the web health monitor with four metric card sections. All values come from the health StateFlow updated by the 5-second polling loop.

Settings Screen: Allows the operator to update the Flask server IP address at runtime. The new URL is applied by calling `NetworkManager.updateBaseUrl()`, forcing a fresh Retrofit client build on the next API call. Toggle switches are provided for Dark Mode and FCM Threat Alerts. A Terminate Console Session button navigates back to Login.

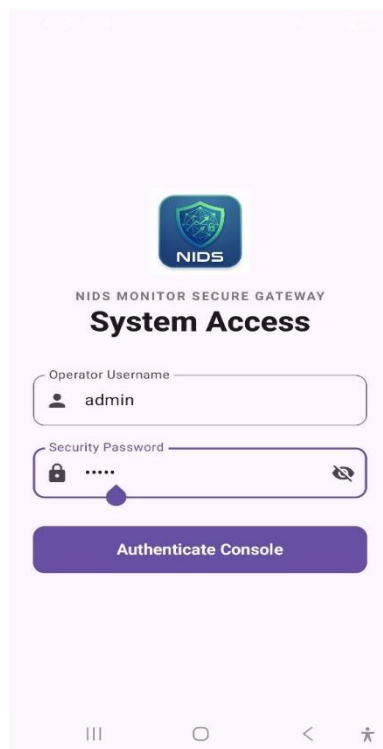


Figure 4.4. Android application login screen

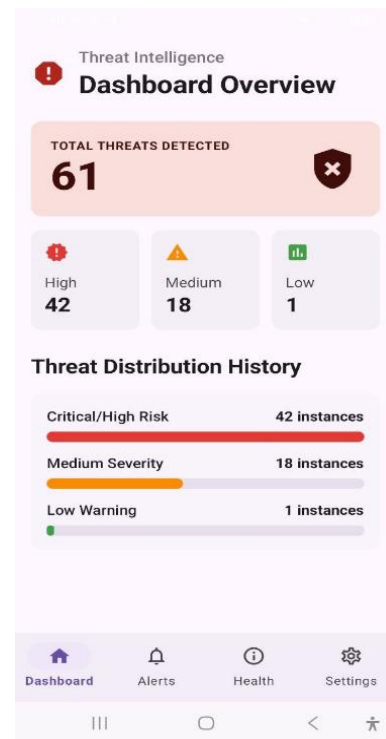


Figure 4.5. Android application dashboard screen.

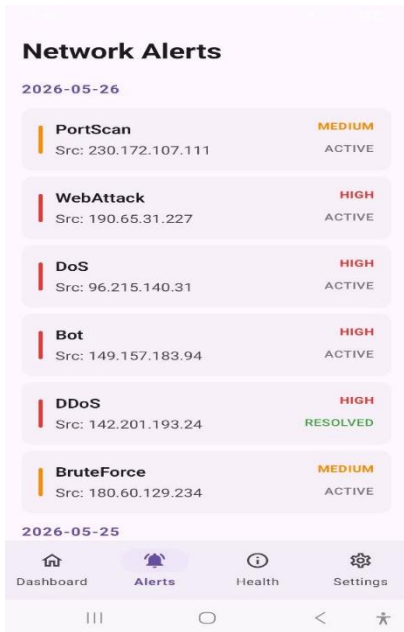


Figure 4.6. Android application alerts screen.

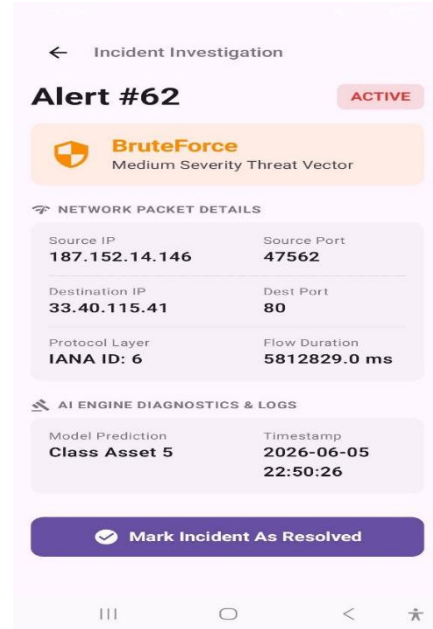


Figure 4.7. Android application alert detail screen.

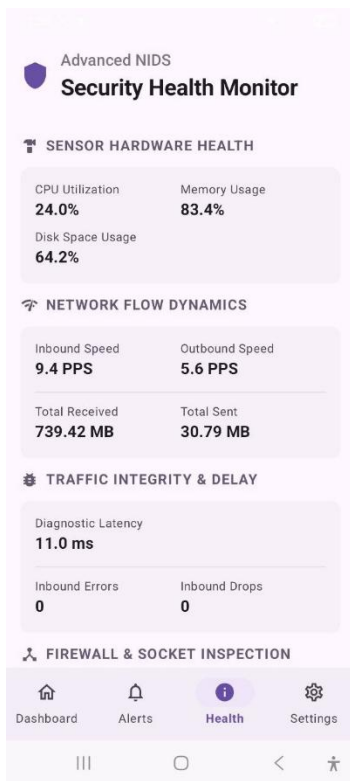


Figure 4.8. Android application health monitor screen.

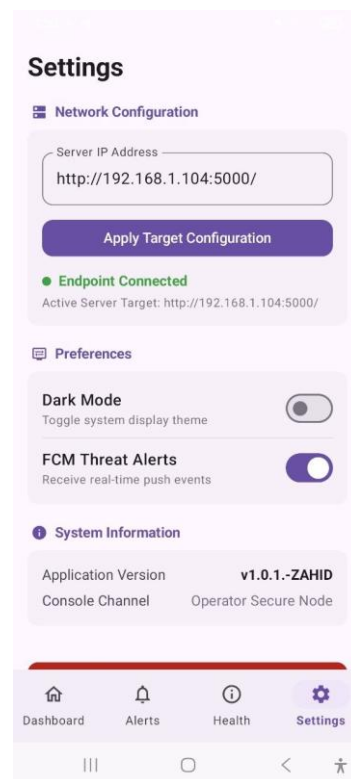


Figure 4.8.1. Android application health setting screen.

4.3. REAL-TIME FCM ALERT NOTIFICATION

When the Flask backend detects an attack and logs the alert record, it immediately sends a Firebase Cloud Messaging notification via the Firebase Admin SDK. On the Android side, push notifications are handled by `NidsMessagingService`, which extends `FirebaseMessagingService`.

In `MainActivity`, the application subscribes to the 'alerts' FCM topic at startup. On Android 13 and above, the `POST_NOTIFICATIONS` runtime permission is requested. When a message arrives, `NidsMessagingService.onMessageReceived()` extracts the notification title and body, reads the `deep_link_uri` from the data payload, creates a high-importance `NotificationChannel`, and builds a notification with a `PendingIntent` pointing to the deep link URI. Notifications are displayed with unique random IDs so multiple simultaneous alerts do not overwrite each other.

When the user taps a notification, Android resolves the `nidsapp://alert/<id>` URI through the `NavHost` deep link and navigates directly to the `AlertDetail` screen for that specific alert ID.

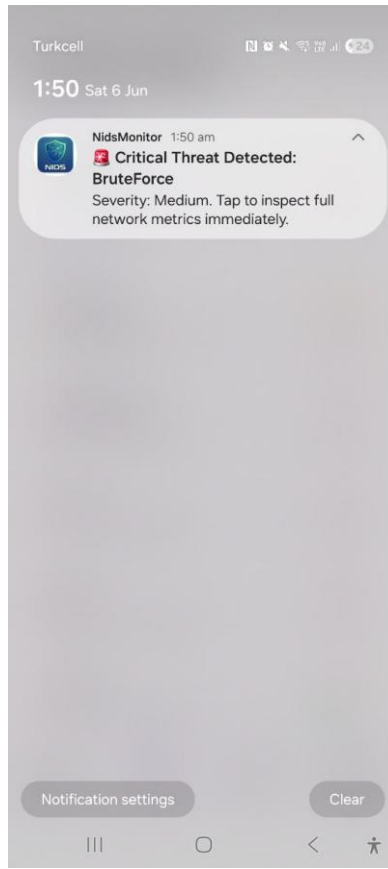


Figure 4.9. FCM push notification displayed on Android device.

4.4. TECHNOLOGY STACK SUMMARY

Table 4.1. Android application technology stack.

Library / Tool	Role
Kotlin	Primary language for all Android development.
Jetpack Compose (Material3)	Declarative UI framework; all screens built as Composable functions.
Jetpack Navigation Compose	Screen routing, back-stack management, and deep link handling.
ViewModel + StateFlow	MVVM architecture; reactive UI state management with coroutine polling.

Library / Tool	Role
Retrofit2 + Gson	HTTP client for consuming Flask REST API; JSON deserialization.
Firebase Cloud Messaging	Receiving and displaying real-time push notifications from Flask backend.
Firebase BOM 33.10.0	Manages consistent Firebase SDK versions across dependencies.
psutil (backend)	System health metrics consumed by the Health screen via /api/health.

CHAPTER 5

CONCLUSION AND EVALUATION

5.1. SUMMARY OF WORK

This project designed and built a Network Intrusion Detection System (NIDS) that detects network attacks in real time using Machine Learning and pushes instant alerts to an Android mobile application. The system was developed end-to-end, covering data preprocessing, model training, backend deployment, web dashboard, and mobile application.

The CICIDS2017 dataset was preprocessed through a five-stage pipeline: combining 8 CSV files, cleaning infinity and NaN values, merging 15 original attack labels into 7 classes, reducing 84 features to 52 through metadata removal and correlation filtering, and performing a stratified 80/20 train-test split with StandardScaler normalization.

Four machine learning classifiers were trained and evaluated. Random Forest and XGBoost both achieved near-perfect classification accuracy on the test set of 565,566 samples. XGBoost was selected for deployment due to its highest macro recall of 99.98% — meaning it missed virtually no attack instances across all seven classes.

The trained model was integrated into a Flask backend that exposes REST API endpoints for simulation, alert retrieval, statistics, health monitoring, and alert resolution. A SQLite database stores the full alert log. Firebase Cloud Messaging delivers real-time push notifications to the Android application when an attack is detected. The web dashboard

provides browser-based monitoring with a Chart.js pie chart, color-coded alert table, and live health metrics updated every 3 seconds.

The Android application, built with Kotlin and Jetpack Compose, provides a complete mobile monitoring interface with Login, Dashboard, Alerts, Alert Detail, Health, and Settings screens. The MVVM architecture with 5-second polling keeps all screens automatically in sync with the backend. FCM deep links let notification taps open the exact alert record in the application.

5.2. MODEL PERFORMANCE RESULTS

Table 5.1. Final model evaluation results.

Model	Accuracy	F1	Recall (macro)	ROC-AUC
Random Forest	99.96%	99.96%	98.84%	1.0000
XGBoost (deployed)	99.93%	99.93%	99.98%	1.0000
LightGBM	99.29%	99.30%	85.68%	0.9915
SVM (binary)	92.99%	93.02%	89.26%	0.9754

XGBoost achieved perfect ROC-AUC (1.0000) and the highest macro recall (99.98%), making it the best model for deployment in a security context where missing an attack is more costly than raising a false alarm. All classes — including minority classes like WebAttack (436 test samples) and Bot (391 test samples) — were classified with precision and recall at or above 0.99.

5.3. LIMITATIONS

Simulation-based traffic: The current implementation samples rows from the pre-collected CICIDS2017 CSV rather than capturing live network packets. Real-time packet capture, feature extraction, and streaming classification remain the primary next step for production deployment.

Dataset age: CICIDS2017 was generated in 2017. Modern attack techniques such as encrypted malware, AI-generated adversarial traffic, and cloud-native attacks are not represented. Retraining on more recent datasets such as CIC-IDS-2018 or UNSW-NB15 would improve coverage.

Single-server deployment: The system runs on a single PC with no load balancing, redundancy, or horizontal scaling. High-volume network environments may exceed the processing capacity of a single Flask server.

Local network only: The Android application requires the mobile device to be on the same local network as the Flask server. Remote monitoring over the internet would need a VPN tunnel, reverse proxy, or cloud deployment.

Binary SVM limitation: The SVM classifier was trained only on binary labels due to computational constraints. It cannot distinguish between attack types and serves only as an attack/no-attack indicator.

5.4. FUTURE WORK

Live packet capture: Replacing the simulation endpoint with a background thread using Scapy or tshark for live traffic capture and CICFlowMeter for feature extraction would make the system fully operational on a real network.

Deep learning models: LSTM or CNN-based models could be evaluated as alternatives to XGBoost for detecting sequential and time-series attack patterns that ensemble methods may miss.

iOS application: Developing a companion iOS application using Swift and SwiftUI would extend the mobile monitoring capability to Apple devices.

Cloud deployment: Deploying the Flask backend on a cloud platform (AWS, Google Cloud, or Azure) with a public IP address would enable remote monitoring from any location without requiring VPN access.

Model retraining pipeline: An automated retraining pipeline that periodically updates the model with newly labeled traffic data would keep detection accuracy high against evolving attack patterns.

Alert correlation: Implementing alert correlation logic to group related alerts from the same source IP or attack campaign would reduce alert fatigue for the operator.

REFERENCES

1. Scarfone, K. and Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94. National Institute of Standards and Technology, Gaithersburg, MD.
2. Lunt, T. F. (1993). A survey of intrusion detection techniques. *Computers & Security*, 12(4), 405–418.
3. Lippmann, R., Haines, J. W., Fried, D. J., Korba, J. and Das, K. (2000). The 1999 DARPA off-line intrusion detection evaluation. *Computer Networks*, 34(4), 579–595.
4. Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. In *Proceedings of the 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pp. 1–6. IEEE.
5. Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116.
6. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
7. Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794. ACM.

8. Kanimozhi, V. and Jacob, T. P. (2019). Artificial intelligence based network intrusion detection with hyper-parameter optimization tuning on the realistic cyber dataset CSE-CIC-IDS2018 using cloud computing. In Proceedings of the International Conference on Communication and Signal Processing (ICCSP), pp. 0033–0036. IEEE.
9. Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A. and Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. IEEE Access, 7, 41525–41550.
10. Yin, C., Zhu, Y., Fei, J. and He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access, 5, 21954–21961.
11. Scarfone, K. and Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94. National Institute of Standards and Technology.
12. Ahmad, I., Abdullah, A. B., Alghamdi, A. S. and Hussain, M. (2012). Artificial neural network approaches to intrusion detection: a review. In Proceedings of the 8th International Conference on Computing Technology and Information Management (ICCTIM).
13. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
14. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems (NIPS), 30, 3149–3157.

15. Google LLC. (2024). Firebase Cloud Messaging documentation. Retrieved from <https://firebase.google.com/docs/cloud-messaging>
16. Google LLC. (2024). Jetpack Compose documentation. Retrieved from <https://developer.android.com/jetpack/compose>
17. Pallets Projects. (2024). Flask documentation (v3.x). Retrieved from <https://flask.palletsprojects.com>
18. Square Inc. (2024). Retrofit documentation. Retrieved from <https://square.github.io/retrofit>
19. OpenAI. (2026). *ChatGPT* (Gemini-era version) [Large language model] <https://chat.openai.com/>.
20. Google. (2026). *Gemini* (v1.5 Pro) [Large language model]. <https://gemini.google.com/>
21. Anthropic. (2026). *Claude* (v4.6 Sonnet) [Large language model]. <https://claude.ai/>

CURRICULUM VITAE

Name Surname : ZAW NAING ZAW NAING

Student Number : 2210213562

Place of Birth : Rakhine/Myanmar

Email : zawnaingtr@gmail.com

EDUCATION

2022 – 2026 Karabük University, Faculty of Engineering, Department of Computer Engineering, Karabük, Turkey.

LANGUAGES

Burmese (Native), English (Advanced), Turkish (Intermediate), Urdu (Intermediate), Rohingya(Native).

TECHNICAL SKILLS

Python, Kotlin, Flask, Jetpack Compose, Machine Learning, Android Development, Firebase, SQLite, HTML/CSS/JavaScript, C, C#, Java, ArcGIS, MongoDB, MySQL

PERSONAL INTERESTS

Network security, mobile application development, machine learning.